Thread-Level Parallelism

CSE4100: System Programming

Youngjae Kim (PhD)
https://sites.google.com/site/youkim/home
Distributed Computing and Operating Systems Laboratory (DISCOS)
https://discos.sogang.ac.kr
Office: R911, E-mail: youkim@sogang.ac.kr

Today

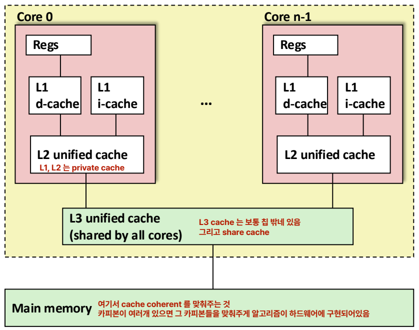
직렬화를 병렬화로 바꾸는
여러개를 작업할 수 있게 해주는
분업화 할 수 있는

- **Parallel Computing Hardware** 하드웨어
  - Multicore
    - Multiple separate processors on single chip
  - Hyperthreading
    - Efficient execution of multiple threads on single core
- **Thread-Level Parallelism** 소프트웨어
  - Splitting program into independent tasks
    - Example: Parallel summation
- **Consistency Models**
  - What happens when multiple threads are reading & writing shared state

Exploiting parallel execution

- **So far, we've used threads to deal with I/O delays**
  - e.g., one thread per client to prevent one from delaying another
- **Multi-core/Hyperthreaded CPUs offer another opportunity**
  - Spread work over threads executing in parallel
  - Happens automatically, if many independent tasks
    - e.g., running many applications or serving many clients
  - Can also write code to make one big task go faster
    - by organizing it as multiple parallel sub-tasks

Typical Multicore Processor



- **Multiple processors operating with coherent view of memory**

Benchmark Machine

- **Get data about machine from /proc/cpuinfo**
- **Shark Machines**
  - Intel Xeon E5520 @ 2.27 GHz
  - Nehalem, ca. 2010
  - 8 Cores
  - Each can do 2x hyperthreading

Example 1: Parallel Summation

- **Sum numbers *0, …, n-1***
  - Should add up to *((n-1)*n)/2*
- **Partition values *1, …, n-1* into *t* ranges**
  - ⌊*n/t*⌋ values in each range
  - Each of *t* threads processes 1 range
  - For simplicity, assume *n* is a multiple of *t*
- **Let's consider different ways that multiple threads might work on their assigned ranges in parallel**

First attempt: `psum-mutex`

- **Simplest approach: Threads sum into a global variable protected by a semaphore mutex.**

```
void *sum_mutex(void *vargp); /* Thread routine */

/* Global shared variables */
long gsum = 0;            /* Global sum */
long nelems_per_thread;   /* Number of elements to sum */
sem_t mutex;              /* Mutex to protect global sum */

int main(int argc, char **argv)
{
    long i, nelems, log_nelems, nthreads, myid[MAXTHREADS];
    pthread_t tid[MAXTHREADS];

     /* Get input arguments */
    nthreads = atoi(argv[1]);
    log_nelems = atoi(argv[2]);
    nelems = (1L << log_nelems);
    nelems_per_thread = nelems / nthreads;
    sem_init(&mutex, 0, 1);
```

psum-mutex.c

`psum-mutex` (cont)

- **Simplest approach: Threads sum into a global variable protected by a semaphore mutex.**

```
    /* Create peer threads and wait for them to finish */
    for (i = 0; i < nthreads; i++) {
        myid[i] = i;
        Pthread_create(&tid[i], NULL, sum_mutex, &myid[i]);
    }
    for (i = 0; i < nthreads; i++)
        Pthread_join(tid[i], NULL);

    /* Check final answer */
    if (gsum != (nelems * (nelems-1))/2)
        printf("Error: result=%ld\n", gsum);

    exit(0);
}
```

psum-mutex.c

`psum-mutex` Thread Routine

- Simplest approach: Threads sum into a global variable protected by a semaphore mutex.

```c
/* Thread routine for psum-mutex.c */
void *sum_mutex(void *vargp)
{
    long myid = *((long *)vargp);          /* Extract thread ID */
    long start = myid * nelems_per_thread; /* Start element index */
    long end = start + nelems_per_thread;  /* End element index */
    long i;

    for (i = start; i < end; i++) {
        P(&mutex);
        gsum += i;
        V(&mutex);
    }
    return NULL;
}
```

psum-mutex.c

`psum-mutex` Performance

- Shark machine with 8 cores, $n=2^{31}$

Threads (Cores)	1 (1)	2 (2)	4 (4)	8 (8)	16 (8)
psum-mutex (secs)	51	456	790	536	681

속도 절감 못하고 있음, 잘못짠거임

- Nasty surprise:
  - Single thread is very slow
  - Gets slower as we use more cores

Next Attempt: `psum-array`

- Peer thread `i` sums into global array element `psum[i]`
- Main waits for theads to finish, then sums elements of `psum`
- Eliminates need for mutex synchronization

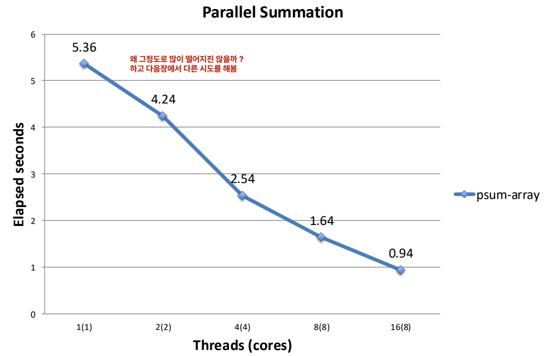
```c
/* Thread routine for psum-array.c */
void *sum_array(void *vargp)
{
    long myid = *((long *)vargp);          /* Extract thread ID */
    long start = myid * nelems_per_thread; /* Start element index */
    long end = start + nelems_per_thread;  /* End element index */
    long i;

    for (i = start; i < end; i++) {
        psum[myid] += i;   lock free 하게 해보자, 한정된 공간을 나눠서 업무 분담해줌
    }
    return NULL;
}
```

psum-array.c

`psum-array` Performance

- Orders of magnitude faster than `psum-mutex`



Next Attempt: `psum-local`

- Reduce memory references by having peer thread i sum into a local variable (register)

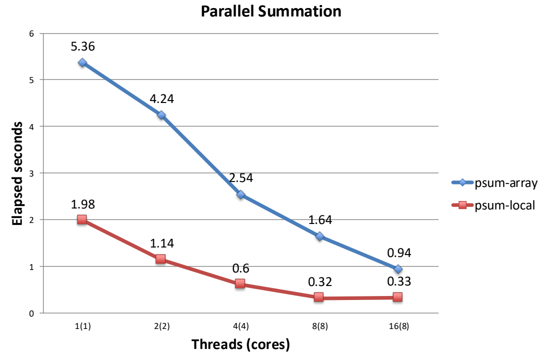
```
/* Thread routine for psum-local.c */
void *sum_local(void *vargp)
{
    long myid = *((long *)vargp);          /* Extract thread ID */
    long start = myid * nelems_per_thread; /* Start element index */
    long end = start + nelems_per_thread;  /* End element index */
    long i, sum = 0;

    for (i = start; i < end; i++) {
        sum += i;  각자 sum 가지고 있는 것으로 바꿔줌 ⇒ sum 을 지역변수로 한것임
                   마지막에 한번만 sum 넣어줌
    }
    psum[myid] = sum;  memory access 횟수를 칸마다 한칸씩만 한것임!
                       전에 한 방법은 메모리를 계속 접근해서 넣어준 것임
    return NULL;
}
```

psum-local.c

`psum-local` Performance

- Significantly faster than `psum-array`



Characterizing Parallel Program Performance

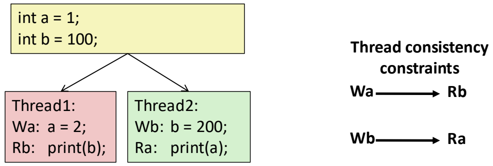
- **p processor cores, T_k is the running time using k cores**
- ***Def.* Speedup: $S_p = T_1 / T_p$**
  - S_p is *relative speedup* if T_1 is running time of parallel version of the code running on 1 core.
  - S_p is *absolute speedup* if T_1 is running time of sequential version of code running on 1 core.
  - Absolute speedup is a much truer measure of the benefits of parallelism.
- ***Def.* Efficiency: $E_p = S_p / p = T_1 / (pT_p)$**
  - Reported as a percentage in the range (0, 100].
  - Measures the overhead due to parallelization

Performance of `psum-local`

Threads (t)	1	2	4	8	16
Cores (p)	1	2	4	8	8
Running time (T_p)	1.98	1.14	0.60	0.32	0.33
Speedup (S_p)	1	1.74	3.30	6.19	6.00
Efficiency (E_p)	100%	87%	82%	77%	75%

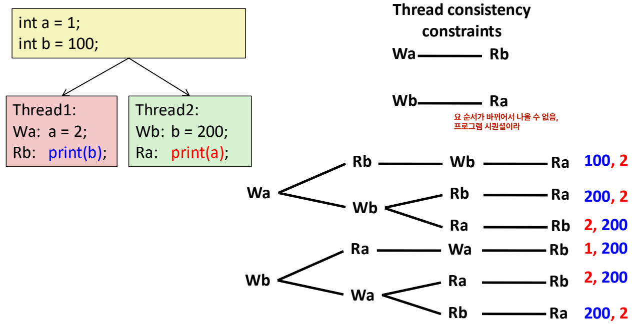
- **Efficiencies OK, not great**
- **Our example is easily parallelizable**
- **Real codes are often much harder to parallelize**

Memory Consistency



- **What are the possible values printed?**
  - Depends on memory consistency model
  - Abstract model of how hardware handles concurrent accesses
- **Sequential consistency**
  - Overall effect consistent with each individual thread
  - Otherwise, arbitrary interleaving

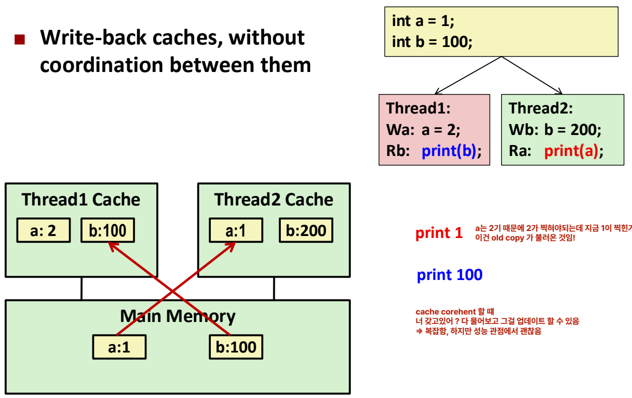
Sequential Consistency Example



- **Impossible outputs**
  - 100, 1 and 1, 100
  - Would require reaching both Ra and Rb before Wa and Wb

Non-Coherent Cache Scenario

- **Write-back caches, without coordination between them**



print 1 a는 2기 때문에 2가 찍혀야되는데 지금 1이 찍힌거임
이건 old copy 가 불러온 것임!

print 100

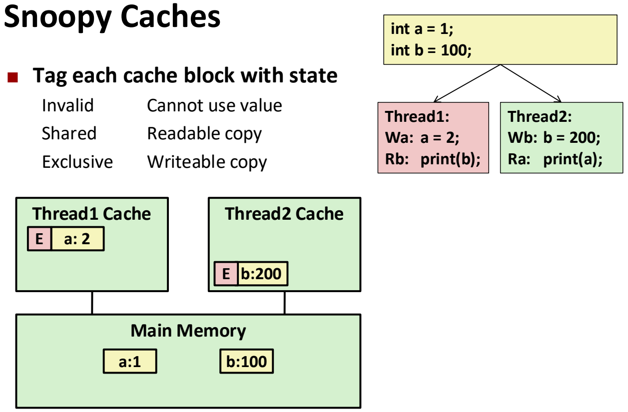
cache corehent 할 때
너 갖고있어 ? 다 물어보고 그걸 업데이트 할 수 있음
⇒ 복잡함, 하지만 성능 관점에서 괜찮음

메모리버스에 주소가 왔다갔다 하는 걸 보고있다가
업데이트가 되었다. 그러면 이제 snoopy 하고 있다가 그거 가지고 있는 애들 다같이 업뎃해주는 것임

Snoopy Caches

- **Tag each cache block with state**

Invalid	Cannot use value
Shared	Readable copy
Exclusive	Writeable copy



Snoopy Caches

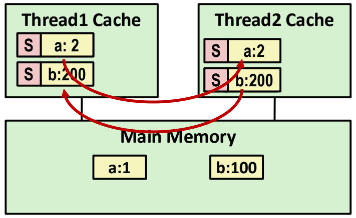
- **Tag each cache block with state**

Invalid	Cannot use value
Shared	Readable copy
Exclusive	Writeable copy

```
int a = 1;
int b = 100;
```

```
Thread1:
Wa:  a = 2;
Rb:  print(b);
```

```
Thread2:
Wb:  b = 200;
Ra:  print(a);
```



print 2

print 200

- When cache sees request for one of its E-tagged blocks
  - Supply value from cache
  - Set tag to S